Repository structure

devops-challenge — the app, cluster config, infrastructure and pipelines change for different reasons, so each has its own folder.

Overview

```
devops-challenge/
├── application/          # Next.js app + Dockerfile
├── k8s/                  # Kubernetes manifests deployed by Argo CD
├── infra/                # Terraform for AWS
├── .github/              # CI/CD workflows
├── architecture/         # diagrams, design notes and enhancements
├── docker-compose.yml    # run the app locally
├── .env.example          # template for local settings
├── README.md             # how to run and deploy
└── ARCHITECTURE.md       # design and decisions
```

Folder by folder

`application/`

```
application/
├── src/
│   ├── app/
│   │   ├── page.tsx, layout.tsx   # the web page
│   │   └── api/health/route.ts    # health check endpoint
│   ├── instrumentation.ts         # startup hook
│   └── lib/logger.ts              # logging
├── public/                        # static files
├── Dockerfile                     # builds the container image
├── .dockerignore
├── package.json, package-lock.json
└── next.config.ts, tsconfig.json, eslint.config.mjs
```

WHY

- The app code and its Dockerfile sit together, so the image is built from this one folder.
- Nothing outside this folder is needed to build the app, which keeps the build small and simple.
- The CI pipeline builds the image from this folder and deploys it on every push to `main`.

k8s/

```
k8s/
├── kustomization.yaml      # lists the manifests + current image tag
├── namespace.yaml
├── serviceaccount.yaml
├── deployment.yaml         # runs the app pods
├── service.yaml
├── ingress.yaml            # exposes the app through a load balancer
├── hpa.yaml                # autoscaling
├── pdb.yaml                # keeps pods available during maintenance
├── networkpolicy.yaml      # limits network access to the pods
├── smoke-test-job.yaml     # checks the app after each deploy
└── platform/               # Argo CD apps: add-ons + the service
    ├── README.md
    ├── argo-cd.yaml
    ├── aws-load-balancer-controller.yaml
    ├── external-dns.yaml
    ├── fluent-bit.yaml
    ├── metrics-server.yaml
    ├── next-bff-service.yaml
    └── values/             # Helm settings for the add-ons
```

WHY

- This folder describes what should run in the cluster. Argo CD keeps the cluster in line with it, so Git is the source of truth.
- One file per Kubernetes resource keeps each change small and easy to review.
- A deploy is just a change to the image tag in `kustomization.yaml`; a rollback is reverting that change.
- `platform/` tells Argo CD what to deploy (the add-ons and the service), kept apart from the service's own manifests.

infra/

```
infra/
├── README.md
├── bootstrap/              # bucket for Terraform state (run once)
├── environments/
│   └── prod/               # the production stack
│       ├── main.tf, variables.tf, outputs.tf, versions.tf
│       ├── backend.hcl.example
│       └── terraform.tfvars.example
├── gitops-bootstrap/       # installs Argo CD in the cluster (run once)
└── modules/
    ├── network/            # VPC and subnets
    ├── eks/                # Kubernetes cluster and nodes
    ├── ecr/                # container registry
    ├── edge/               # DNS, TLS certificate, firewall
    ├── log-archive/        # S3 bucket for logs
    ├── observability/      # alarms and notifications
    ├── ci-identity/        # access for CI to push images
    ├── ci-terraform/       # access for CI to run Terraform
    └── workload-identity/  # AWS access for pods
```

WHY

- Split into three stacks by when they run: `bootstrap` creates the state bucket first, `prod` builds the environment, and `gitops-bootstrap` installs Argo CD once the cluster exists.
- Each module does one job, which makes it easier to read, change and reuse.
- `environments/` only combines modules, so adding another environment means a new folder, not copied code.
- Only `.example` config files are committed; real values and state stay out of Git.

.github/

```
.github/
├── workflows/
│   ├── app-ci.yml          # build, scan and publish the app image
│   ├── infra.yml           # plan and apply Terraform
│   └── security.yml        # scan for leaked secrets
├── scripts/
│   ├── update-image-tag.sh
│   └── plan-comment.sh
└── dependabot.yml          # monthly, grouped dependency updates
```

WHY

- One workflow per area: app changes and infra changes run separate pipelines.
- Longer steps live in `scripts/`, so the workflow files stay short and the scripts can be run locally.

architecture/ and root files

```
architecture/
├── README.md                                # flows and trust boundaries
├── infrastructure.png, .drawio              # AWS: VPC, 3 AZs, EKS, edge, IAM roles
├── k8s-components.png, .drawio              # cluster: namespaces, workloads, Argo CD
├── application-pipeline-activity.png, .drawio
│                                            # PR → CI → ECR → Argo CD → rollback
├── repo-structure.pdf                       # this document
└── enhancements/                            # what's not built yet, and how to improve it
    ├── room-for-improvement.md              # known gaps and trade-offs
    ├── backup-and-restore.md                # recovery runbook
    └── chaos-test-plan.md                   # failure experiments to run

README.md, ARCHITECTURE.md, docker-compose.yml, .env.example, .gitignore
```

WHY

- Three diagrams, one per question: *where it runs* (infrastructure), *what runs in the cluster* (k8s components) and *how a change ships* (application pipeline). Each PNG has its editable draw.io source next to it.
- The diagrams sit next to the written notes, so the pictures and the text that explains them are reviewed together.
- `enhancements/` keeps the follow-up material (known gaps, the backup and restore runbook, the chaos test plan) apart from the description of what is built.
- The root only has entry points: README for how to run it, ARCHITECTURE.md for the design.
- `docker-compose.yml` runs the same image locally that runs in the cluster.